



Construcción de software y toma de decisiones (Gpo 501)

**Competencia STC205: Elaboración de casos
de prueba de software**

Carlos Arturo Gómez Ayala - A01711027
ITESM Campus Querétaro
Profesor: Enrique Alfonso Calderón Balderas
Co-Titular: Alejandro Fernández Vilchis
Co-Titular: Denisse Lizbeth Maldonado Flores

Fecha: 1/06/2026

Elaboración de Pruebas de Software

* Las Pruebas de Software son 30-50% del costo\$

¿Qué es una Prueba?

Es una verificación automática de que tu código produce el resultado esperando para una entrada.

Si la salida real coincide lo esperado ✓✓

Si no pasa la prueba → Falla y revela defecto

Rational Unified Process (RUP): Mejor comenzar a probar lo más pronto ASAP posible.

[Use Cases → Test cases]

Como se sabe los casos de uso determinan y definen los requerimientos funcionales del Software.

RUP: "Los casos de uso describen completamente la secuencia de acciones hechas por el sistema para dar un resultado observable resultando en **valor** para una persona o a otro sistema usando el producto."

1erº Fundamental

2ºdo

Test cases → Test Precedimientos → Test Scripts ^{td.}

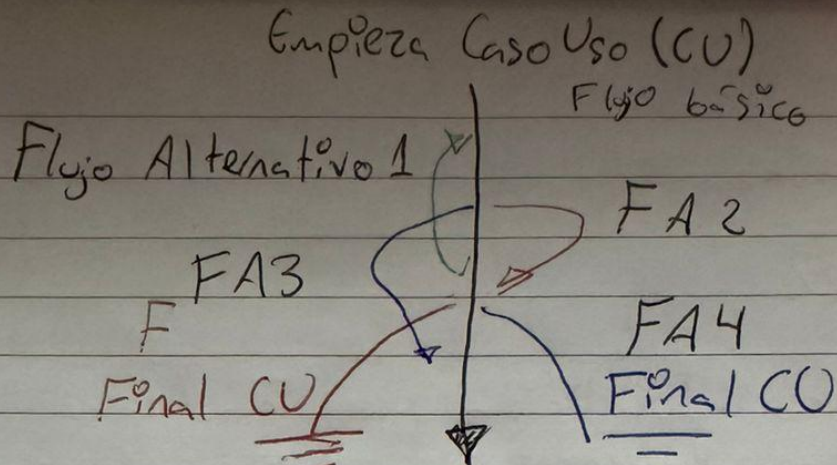
¿Por qué?

Test Cases: Identifican y comunican las condiciones que se van a implementar y son necesarias para verificar exitosas y aceptables implementaciones de los requerimientos productos.

Tabla Formato Use Cases

Nombre	Apropiado
Breve Descripción	Rol y Propósito
Flujo de Eventos	Lo que hace en ese caso de uso, no como resuelve específicamente. La debe entender el cliente
Requerimientos Special	Colecta todos los RF, RNF, que no se consideran en el caso de uso, pero de los que se debe considerar para diseño o implementación
Precondiciones	Lo que se necesita antes que empiece el caso de uso
Post Condición	

Lo más Importante: Flujo de Eventos.

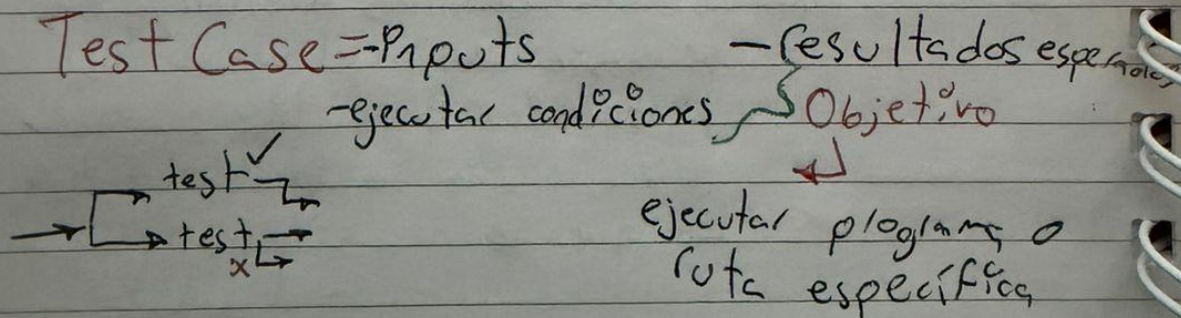


Ejemplos de Enroll en Iris Tec Monterrey.

Tabla Escenarios Casos de Uso (CU)

Escenario 1	Basic Flow
Escenario 2	Basic Flow Alternativa 1
Escenario 3	Basic Flow Alternativa 2 y 1
Escenario 8	Basic Flow Y/O Alternativa 1 a la 8, etc.

Generar Test Cases



RECORDAR: El propósito es de identificar y comunicar condiciones que se van a implementar.

Test cases son necesarios para exitosas y aceptables implementaciones de los requerimientos.

Términos a usar

Error: Acción humana incorrecta (Programador, tester, etc).

Defecto: Bug en el código que resulta del error.

Falla: Lo que el usuario observa cuando ejecuta.

Caso de Prueba: Una entrada específica + el resultado que se espera al ejecutar.

Test suite: Conjunto casos relacionados (Ej. todas las pruebas de personaje).

Oráculo: Criterio que decide si el resultado es correcto o no.

Automatización: Que las pruebas se ejecuten solas, no con humanos.

E2E/UI

Pruebas de Integración

Pruebas Unitarias

& Más

Unitarias: Una función o clase aislada.

Integración: Varios módulos trabajando juntos.

Sistema E2E: Sistema completo extremo a extremo.

Aceptación: Verifica que cumple los requisitos del cliente.

Usabilidad: Personas reales usando el producto.

Regresión: Que no rompió lo que ya funcionaba.

Verificación vs Validación

Verificación	Validación
¿Construimos el producto correctamente? el código funciona?	¿Construimos lo que quiere el cliente? el producto correcto?
- Unitarias - Integración - P. sistemas	- P. usabilidad. - P. en voz alta. - P. aceptación

Anatomía Caso Prueba

Campo	¿Qué es?	Ejemplo
ID	Identificador	TC-001
Descripción	¿Qué prueba?	Recibir daño = Vida ↓
Precondición	Estado inicial necesario	Personaje vida = 100
Entrada	Datos que se le dan	Daño = 30
Acción	Operación a ejecutar	personaje recibir daño (30)
Salida esperada	Resultado	vida = 70

Clases de Equivalencia

Agrupar entradas que se traten igual. Si una funciona las demás también

X	✓	X
< 18 años	$18 \leq \text{edad} \leq 130$	$\text{edad} > 130$
no vota	válido a votar, etc entre esos datos	

3 casos de Prueba cubren toda la lógica

Antes de este curso hacía lo que creo que hace todo el mundo cuando esta programando y era que imprimía valores, mandaba peticiones y abría el navegador para ver si algo explotaba o no funcionaba o que se imprimía. Eso funciona para detectar errores obvios y finalmente era prueba y error, pero tiene un problema fundamental y es que no es repetible y algo que se me ha quedado mucho de la clase, es ser metódico para no tener que estar arreglando errores 'quirúrgicamente' después, costando recursos y costando tiempo. Si modifico una función hoy, mañana tendría que volver a hacer todo eso manualmente para saber si sigo sin romper nada y al día siguiente otra vez y así. Además, ese conocimiento de qué probé vive solo en mi cabeza, mi equipo no lo ve y no hay registro de ello, entonces nadie sabría que falla o fallaría sin pruebas mas que la persona que lo hizo.

Error, defecto y falla

Estas tres palabras parecen sinónimos pero no lo son. Un error es la acción humana incorrecta, lo que hizo el programador mal, por ejemplo escribir `vidaActual + daño` en lugar de restarle. Ese error produce un defecto, que es el bug concreto que queda en el código esperando ser ejecutado. Y cuando alguien finalmente ejecuta ese código y ve algo raro en pantalla eso es la falla, lo que el usuario observa.

Verificación vs. validación

Esta distinción me parece de las más importantes del curso. Verificación pregunta si construimos el producto correctamente, mira hacia adentro ¿el código hace lo que dice que hace? Ahí caen las pruebas unitarias, de integración y de sistema. Validación pregunta si construimos el producto correcto, mira hacia afuera: ¿es lo que el usuario realmente necesita? Ahí caen las pruebas de usabilidad, las de pensamiento en voz alta y las de aceptación, ambas son importantes para llevar una programación exitosa y un producto de valor al cliente.

La pirámide de pruebas

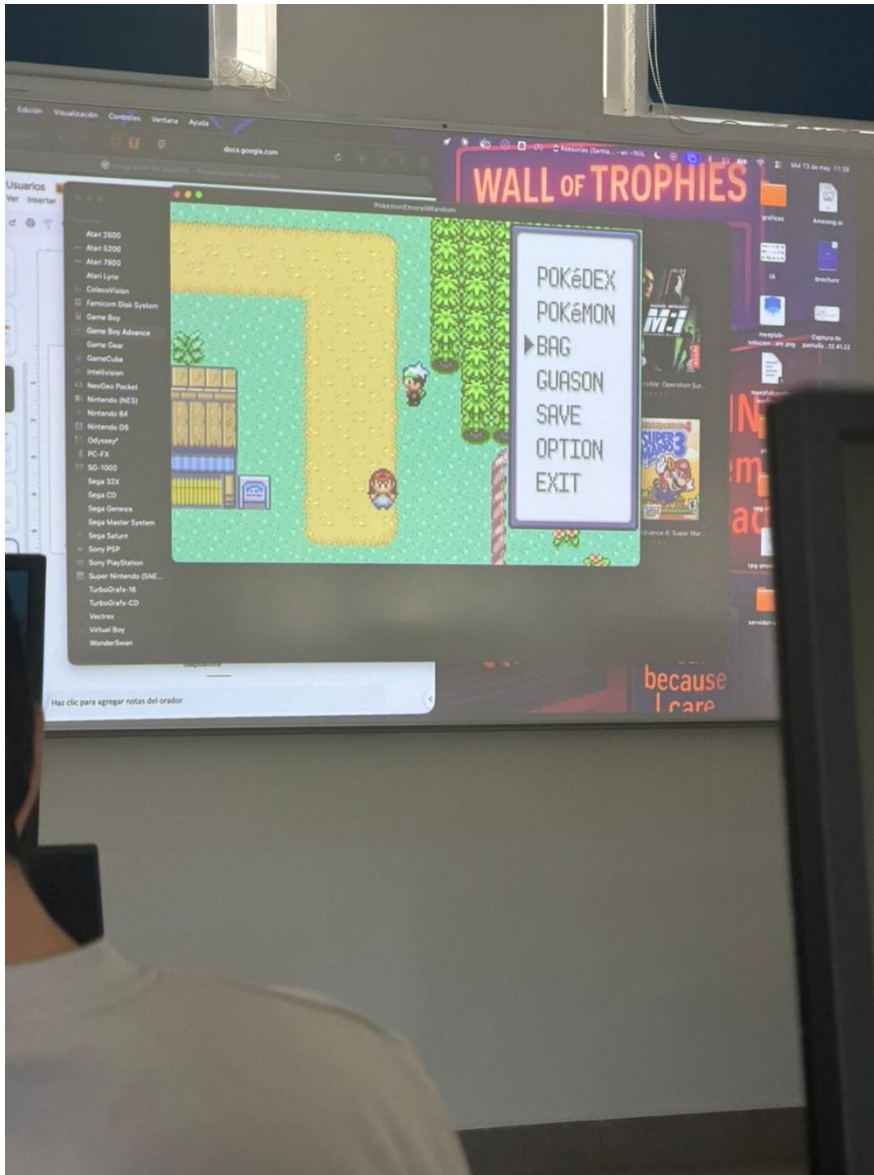
en la base van muchas pruebas unitarias porque son rápidas, baratas y aíslan defectos con precisión. En el medio van las de integración. Y en la punta, pocas pruebas E2E porque son lentas, frágiles y cuando fallan no siempre te dicen dónde está el problema exactamente.

Anatomía de un caso de prueba

Antes de tocar cualquier framework hay que saber qué probar. Un caso bien diseñado tiene seis campos: un ID para identificarlo, una descripción de qué se está probando, una precondition que define el estado inicial necesario, la entrada

con los datos que se le pasan, la acción que se va a ejecutar y la salida esperada. Sin esos seis elementos, un caso de prueba es ambiguo y difícil de repetir.

Pruebas de pensamiento en voz alta (Think Aloud)



Esta no la olvido por nada del mundo y que jugazos pusieron de Mario y Pokemon Emerald con un mod randomizer de Pokemon. En fin, esta es una técnica de validación, no de verificación. Mientras el usuario usa el sistema, va hablando en voz alta diciéndote qué piensa, qué espera que va a pasar y qué le confunde o como va progresando, etc. No intervienes, solo observas y tomas notas como el ejemplo de clase. El punto no es probar que el código funciona, que eso ya lo cubren las unitarias, sino descubrir si la interfaz y el flujo tienen sentido para alguien que no diseñó el sistema.

Lo que más te revela es exactamente lo que el usuario dice antes de cometer un error, porque ahí está la brecha entre lo que tú diseñaste y lo que él entiende. Si alguien dice 'supongo que este botón hace X' y hace clic esperando X y le pasa Y, eso es un hallazgo de usabilidad que ninguna prueba automatizada te daría, como el botón de salto cambiando de letra, pero siendo el botón de abajo y que se cambie ahora al botón de arriba de 4 botones. El punto es ver y documentar esas interacciones.

Pruebas de integración

Una prueba unitaria verifica una función aislada del resto. Una prueba de integración verifica que varios módulos funcionan correctamente cuando trabajan juntos. La diferencia importa porque dos módulos pueden pasar sus pruebas unitarias por separado y aun así romperse cuando se llaman entre sí ya sea por incompatibilidades en los datos que se pasan, por estados compartidos que no se limpian entre pruebas o por dependencias externas como las bases de datos.

Valores frontera

Los bugs viven en los bordes. El error más clásico del mundo es escribir $<$ cuando debería ser $\leq$ o viceversa. Por eso la técnica de valores frontera dice que hay que probar justo abajo del límite, exactamente en el límite y justo arriba del límite. Si la frontera es 18, pruebo 17, 18 y 19, no 5 y 50.

Aplicación al proyecto CoreData

El proyecto en el que estamos trabajando es un sistema web de cumplimiento PLD desarrollado para SVA LAW, una firma legal fintech que opera como SOFOM bajo regulación CNBV. El sistema maneja el flujo operación → alerta → reporte y tiene más de veinte requerimientos funcionales implementados en Node.js con Express, PostgreSQL y despliegue en Vercel.

En términos de las pruebas, la actividad de integración que documentamos fue una revisión funcional de las vistas principales del sistema. Evaluamos el menúel dashboard, el módulo de operaciones, el de alertas y el listado de clientes. El criterio fue si cada sección mostraba los datos correctos, si los controles respondían como se esperaba y si había consistencia visual en colores y tipografía y demás. Los resultados mostraron que funcionalidades como el reordenamiento de columnas y la paginación de clientes operaban correctamente, mientras que otras como los filtros de operaciones y la lista de personas bloqueadas aún estaban pendientes por implementarse o que no pudiéramos colocar esos filtros donde quisiéramos.

Esta revisión es un ejemplo concreto de pruebas manuales de integración a nivel de sistema, donde verificas que los módulos del frontend y del backend funcionan en conjunto para el usuario final. Complementa las pruebas unitarias que se hicieron con Jest sobre los módulos de lógica de negocio. Los conceptos de clases de equivalencia y valores frontera son aplicables al sistema por ejemplo, en la validación del RFC de un cliente, la clase válida tiene un formato específico de 13 caracteres alfanuméricos según el SAT, la clase inválida incluye formatos distintos y los valores frontera son cadenas de 12 y 14 caracteres para detectar errores de conteo. El mismo razonamiento aplica a campos como CURP, montos de operación y fechas, todos con reglas precisas de la regulación CNBV que hacen que los casos de prueba puedan derivarse directamente de los requisitos.

Por lo que, se deben de hacer los casos de prueba o de test lo mas pronto posible como menciona el artículo, ir de la mano de requerimientos al mismo tiempo las pruebas, ahorraría bastante tiempo y trabajo para después. Se cometen menos defectos o fallas si no se cometen errores y da como resultado un mejor producto porque se tuvo una metodología previa y se ejecuta de mejor manera.

Read the use-case textual description and identify each combination of main and alternate flows -- the scenarios -- and create a scenario matrix. Table 3 shows a partial scenario matrix for the Register for Courses use case. This is a simple example with no nested alternate flows.

Table 3: Partial Scenario Matrix for the Register for Courses Use Case

Scenario Name	Starting Flow	Alternate

Scenario 1 - Successful registration	Basic Flow	
Scenario 2 - Unidentified student	Basic Flow	A1
Scenario 3 - User quits	Basic Flow	A2
Scenario 4 - Course catalog system unavailable	Basic Flow	A4
Scenario 5 - Registration closed	Basic Flow	A5
Scenario 6 - Cannot enroll	Basic Flow	A3

Referencia: Generating Test Cases From Use Cases by Jim Heumann.
Requirements Management Evangelist Rational Software
Material de clase.